

SZÁMÍTÓGÉP ARCHITEKTÚRÁK

Mikroarchitektúra szintje II.

Gyorsítási lehetőségek

Tanács Attila

Mikroarchitektúra szint tervezése

- **MIC-1**

- Fiktív mikroarchitektúra
- „Közepesen” egyszerű, „közepesen” gyors
- Kb. 5000 tranzisztorból felépíthető lenne (memória nélkül)

- **Sebesség vagy ár**

- A részegységek gyorsabb vagy lassabb működésre tervezhetők
- Különböző költséggel

- **Gyorsítási lehetőségek**

- Csökkenteni egy utasítás végrehajtásához szükséges óraciklusok számát
 - Külön áramkör a **PC** regiszter növelésére (ne az ALU végezze)
- Egyszerűbb felépítés a rövidebb óraciklushoz
- Utasítások átlapolt végrehajtása
 - Ne legyenek üres ALU ciklusok (memóriabeolvasásra várva)
 - Utasításbetöltő áramkörök leválasztása a fő adatútról
 - ...

Végrehajtási út hosszának csökkentése

- **Értelmező ciklus és a mikrokód összefűzése**

- Meddő ciklusok kihasználása
 - A lassú elérésű központi memória miatt ne legyen annyi üres várakozás
- Példa a **POP** műveleten keresztül

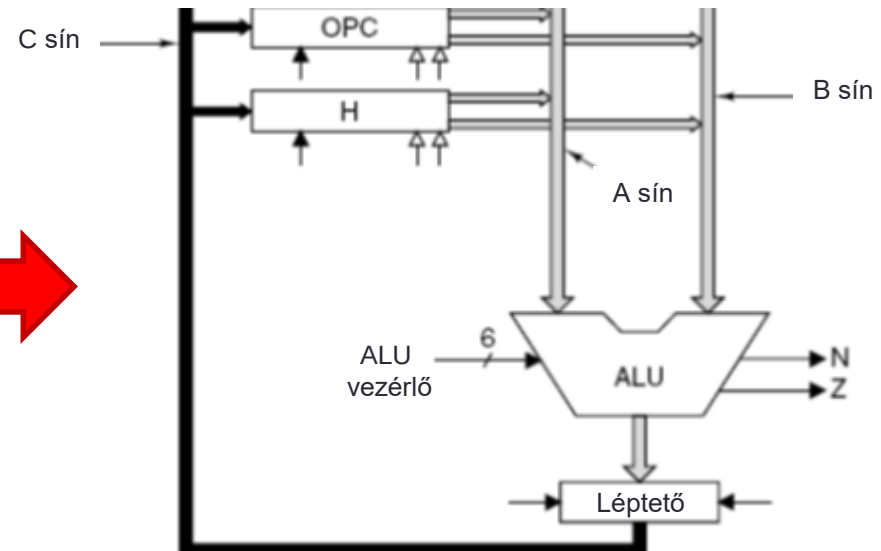
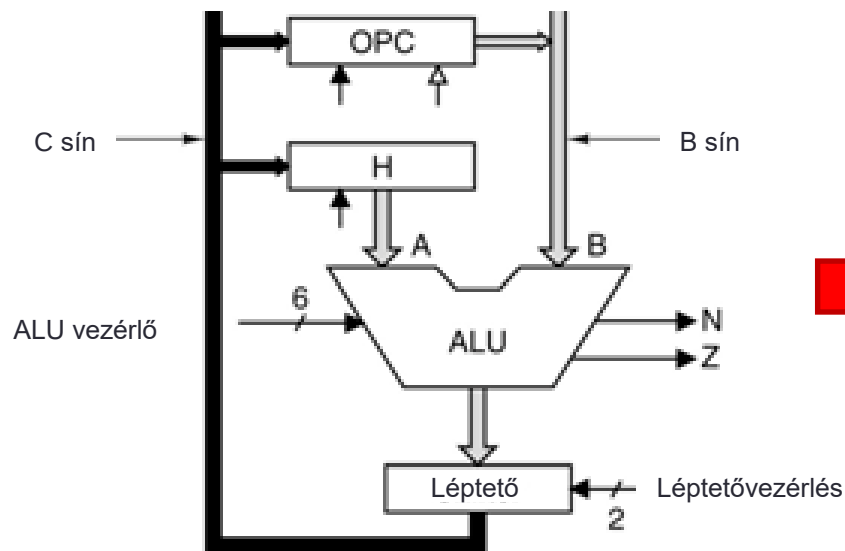
Címke	Műveletek	Megjegyzések
pop1	MAR = SP = SP - 1; rd	Verem teteje alatti szó olvasása
pop2		Várakozás a beolvasásra
pop3	TOS = MDR; goto Main1	Beolvasott szó a TOS-ba (<i>Top of Stack</i>)
Main1	PC = PC + 1; fetch; goto (MBR)	Következő utasítás végrehajtására ugrik

Címke	Műveletek	Megjegyzések
pop1	MAR = SP = SP - 1; rd	Verem teteje alatti szó olvasása
Main1.pop	PC = PC + 1; fetch	Következő utasítás beolvasása, míg a memóriaolvasás zajlik
pop3	TOS = MDR; goto (MBR)	Beolvasott szó a TOS-ba

Végrehajtási út hosszának csökkentése

• Többsínes architektúra

- Háromsínes példa
- Az ALU **A** regisztere közvetlenül kaphat értéket, nem csak **H**-ból
- Tetszőleges regiszterek összeadhatók egy cikluson belül
- Nem kell külön adatútciklus a **H**-ba töltéshez



Végrehajtási út hosszának csökkentése

- **Utasításbetöltő egység**

- IFU – *Instruction Fetch Unit*
- Tehermentesítsük az ALU-t

- **IFU feladatai**

- PC növelése
 - Teljes összeadó helyett elég egy növelő áramkör
- Bájtok betöltése a memóriából, mielőtt felhasználásra kerülnek
- 8 és 16 bites operandusok előkészítése (összerakása)

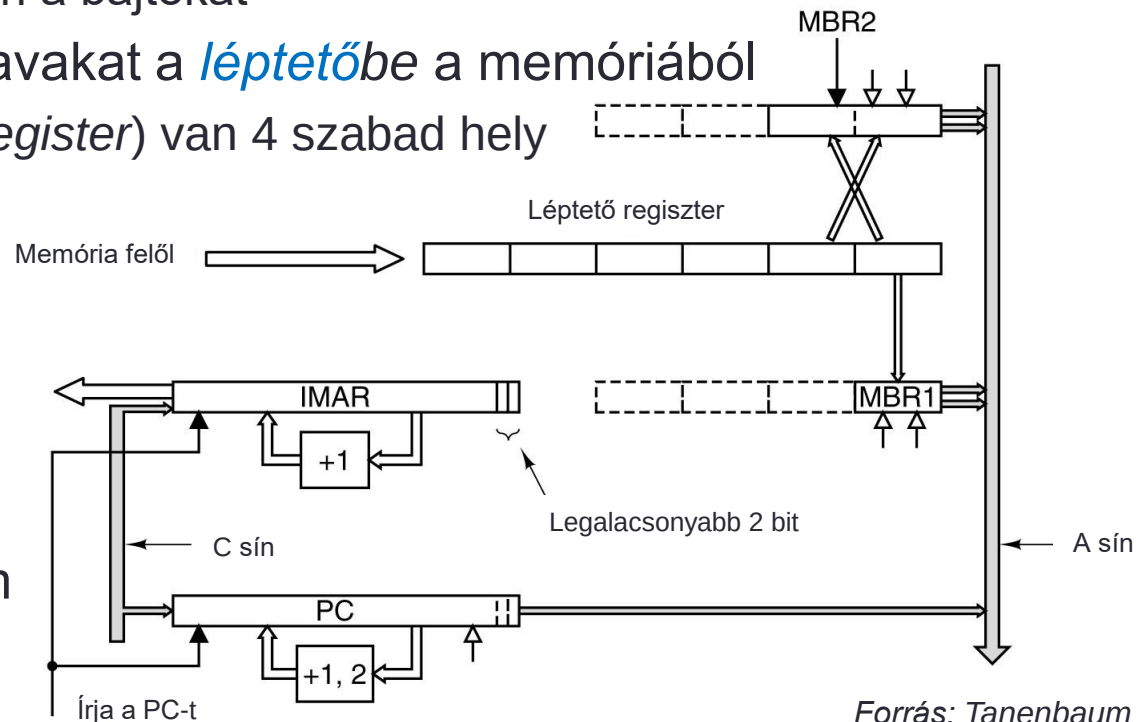
- **Működési módok**

- Mindig legyen elérhető a következő 8 és 16 bites adat (műveleti kód értelmezés nélkül)
- Műveleti kódok értelmezése, a megfelelő számú bájtt beolvasása, regiszterben tárolása

Utasításbetöltő egység (IFU)

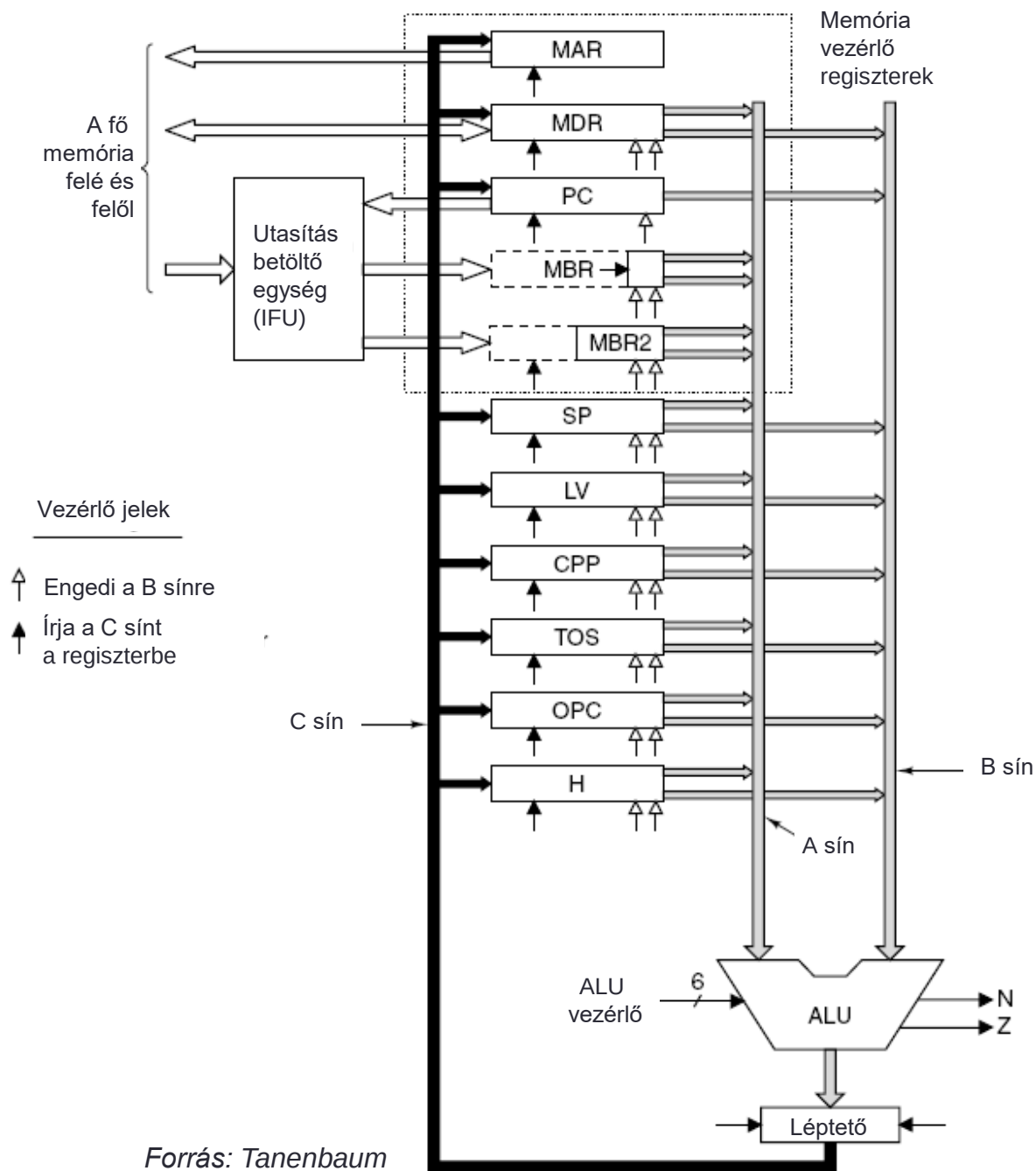
Újdonságok

- **MBR** helyett **MBR1** (8 bites) és **MBR2** (16 bites)
 - Soron következő bájt(ok), közvetlenül elérhető mindkettő
- **MBR1** és **MBR2** kiolvasását érzékeli
 - Jobbra tolja a léptetőben a bájtokat
- Előre betölt 4 bájtos szavakat a *léptetőbe* a memóriából
 - Ha a léptetőben (*shift register*) van 4 szabad hely
 - Blokkolás
 - Ha **MBR2** kellene, de még olvas a memóriából
- **PC** módosulásakor frissítés
- **IMAR**: saját memóriacím regiszter



MIC-2

- Az eddig tárgyalt gyorsítások alkalmazása
- Csökken az utasítások végrehajtásához szükséges mikroutasítás-szám
 - Utasításonként különböző mértékben
- Gyorsabb gép, valamivel magasabb áron



Csővonalas megvalósítás (MIC-3)

• Adatút felosztása

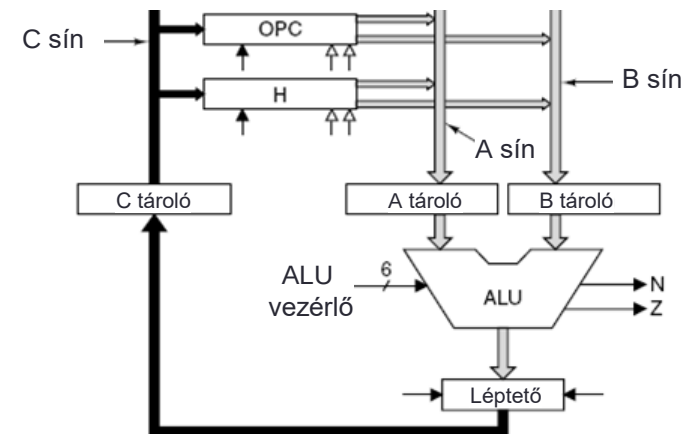
- Párhuzamosan működni tudó részekre
- Sínenként 1-1 **tároló** közbeiktatásával
- Tároló nélkül: A síneken a feszültségszinteket meg kell tartani.
- A tároló a bemeneti szintet „megjegyzi”, pl. az ALU működése közben az **A** és **B** sínek már más feszültségszintet kaphatnak.

• Adatútciklus időigénye korábban

- Regiszterek **A** és **B** sínre
- ALU és a léptető regiszter munkája
- Eredmények regiszterekbe írása

• Felosztás után

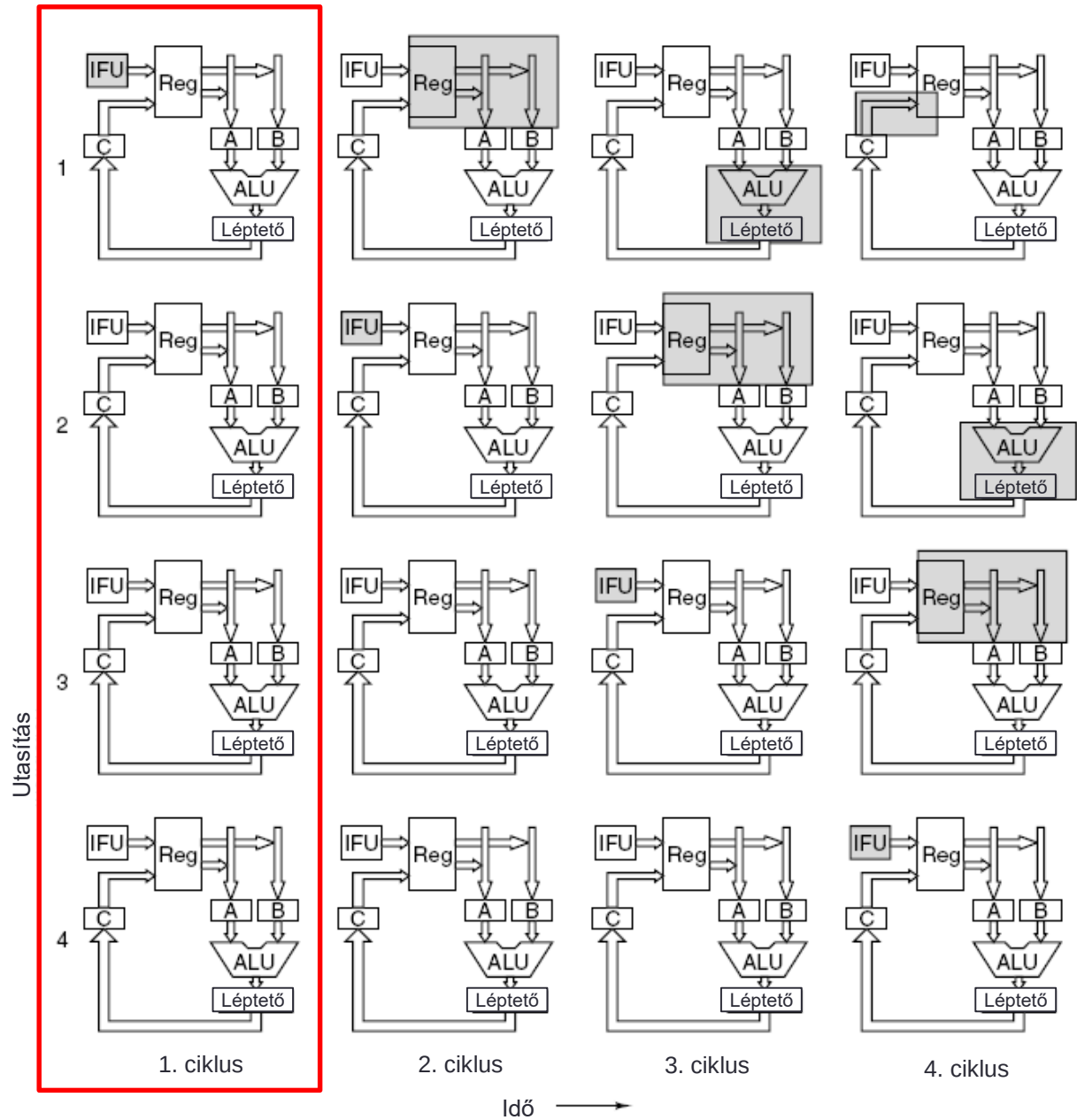
- Gyorsíthatjuk az órát, mert a maximális késleltetés kisebb
- Minden ciklus alatt az adatút minden része dolgozhat
 - **Mikrolépések:** **A** és **B** betöltése; művelet elvégzése (ALU); **C** feltöltése és regiszterekbe írás (közben az IFU tölti a következő utasítást)
 - **Függőségek kialakulhatnak!**



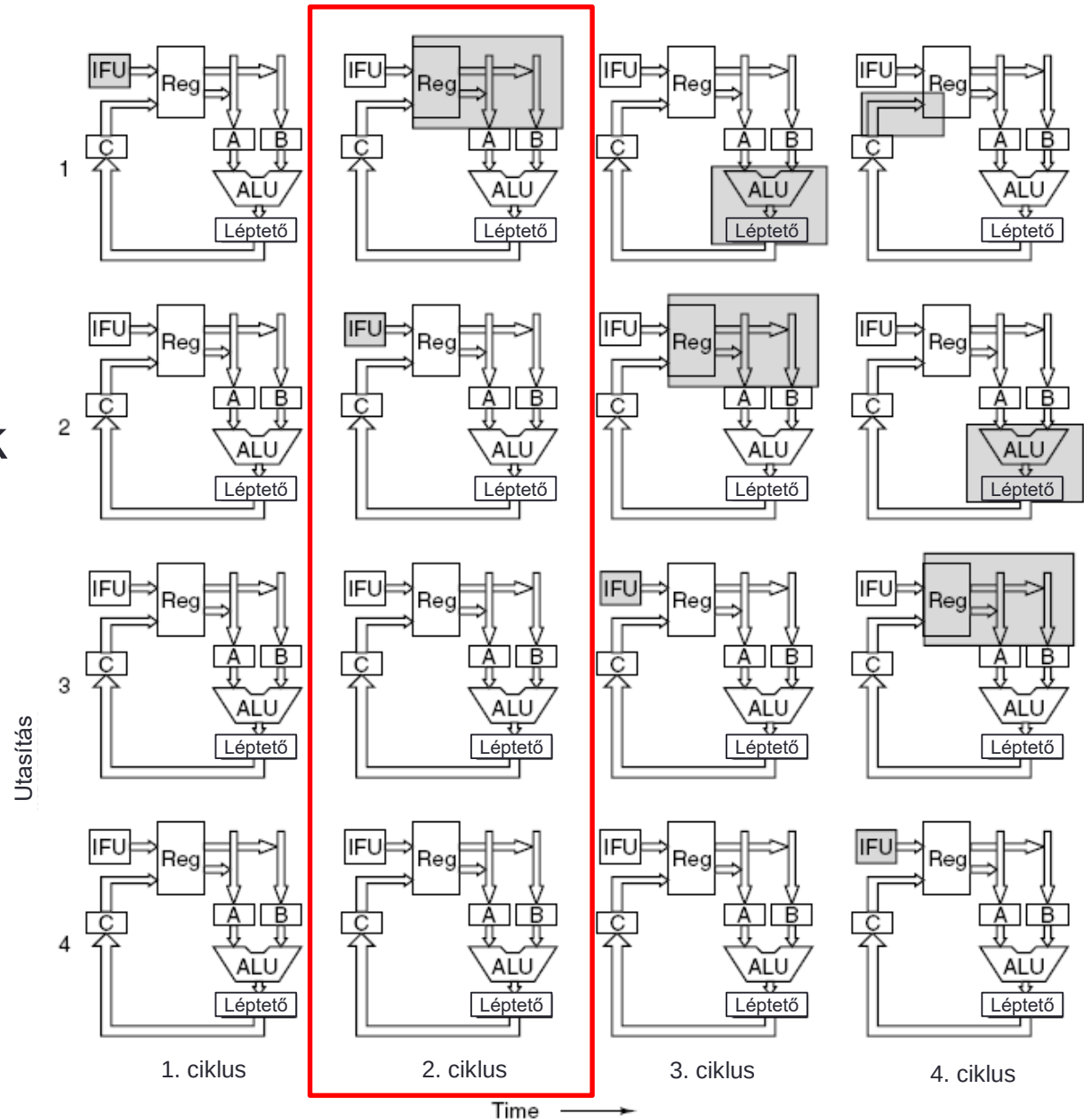
Forrás: Tanenbaum

- IFU betölti az első utasítást

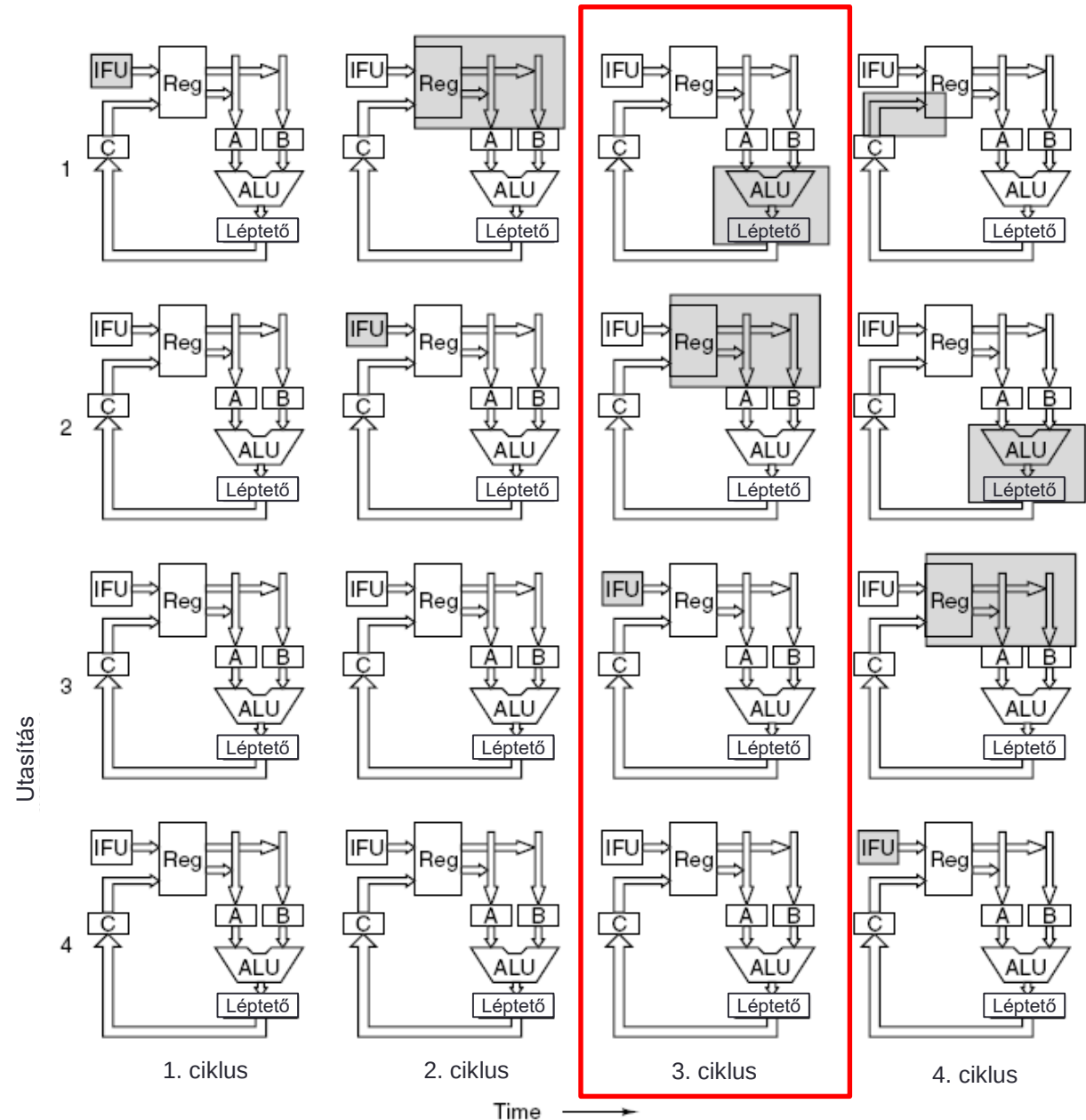
Megjegyzés: ez egy általános csővezeték végrehajtást mutat, ezért nem jelenik meg a végrehajtás során esetlegesen előálló utasítás blokkolás.



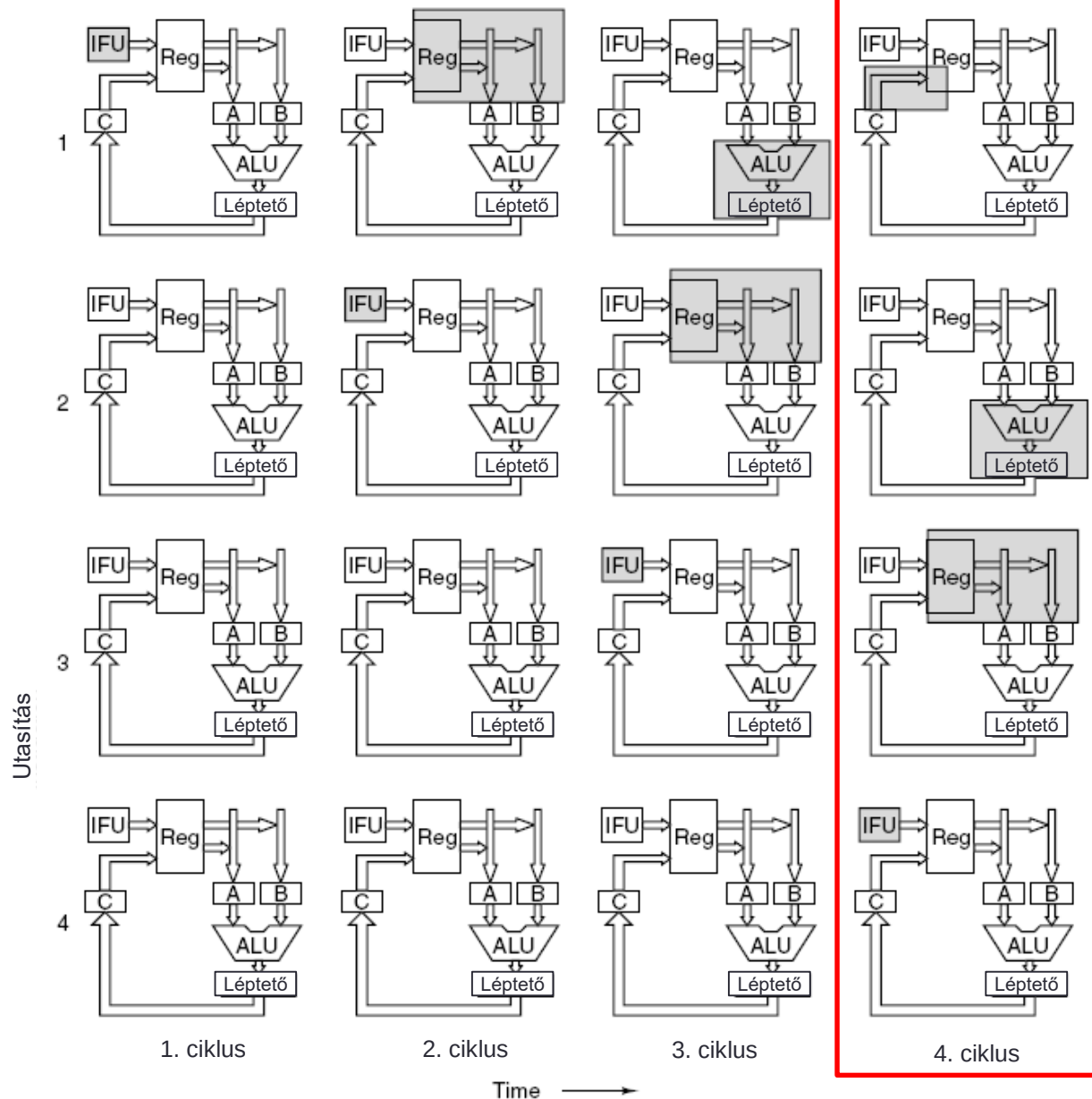
- Az első utasítás által kért regiszterek **A** és **B** tárolóba töltődnek
- IFU betölti a második utasítást



- Az első utasítás az ALU-t használja
- Az A és B tároló a második utasításnak megfelelően töltődik fel
- IFU betölti a harmadik utasítást



- Az első utasítás eredménye tárolódik
- A második utasítás használja az ALU-t
- A harmadik utasításnak megfelelő regiszterek **A** és **B** tárolókba kerülnek
- IFU betölti a negyedik utasítást



Csővonalas megvalósítás (MIC-3)

- **SWAP** utasítás példa
 - Verem tetején lévő két érték megcserélése
 - MIC-2 architektúrán **6 ciklus**

Címke	Műveletek	Megjegyzések
swap1	$MAR = SP - 1; rd$	Verem teteje alatti szó olvasása MDR -be
swap2	$MAR = SP$	Cím a verem tetejére áll
swap3	$H = MDR; wr$	MDR mentése, szó írása a verem tetejére
swap4	$MDR = TOS$	Előző TOS az MDR -be kerül
swap5	$MAR = SP - 1; wr$	Előző TOS a verem második helyére
swap6	$TOS = H; goto (MBR1)$	TOS frissítése az új értékre; ugrás a következő utasítás végrehajtására

Csővonalas megvalósítás (MIC-3)

- **SWAP** utasítás mikrolépései MIC-3-on
 - 11 ciklus, de 1 ciklus sokkal rövidebb, mint MIC-2-n
 - Utasítások mikrolépéseinek párhuzamos végrehajtása
 - Mikrolépés sorozat:

	swap1	swap2	swap3	swap4	swap5	swap6
Ciklus	MAR = SP - 1; rd	MAR = SP	H = MDR; wr	MDR = TOS	MAR = SP - 1; rd	TOS = H; goto (MBR1)
1	B = SP					
2	C = B - 1	B = SP				
3	MAR = C; rd	C = B				
4	MDR = Mem	MAR = C;				
5			B = MDR			
6			C = B	B = TOS		
7			H = C; wr	C = B	B = SP	
8			Mem = MDR	MDR = C	C = B - 1	B = H
9					MAR = C; wr	C = B
10					Mem = MDR	TOS = C
11						goto (MBR1)

Csővonalas megvalósítás (MIC-3)

• 1. ciklus

- **swap1** mikroutasítás végrehajtása elindul
 - 1. mikrolépés (veremmutató értéke a **B** sínre)

	swap1	swap2	swap3	swap4	swap5	swap6
Ciklus	MAR = SP - 1; rd	MAR = SP	H = MDR; wr	MDR = TOS	MAR = SP - 1; rd	TOS = H; goto (MBR1)
1	B = SP					
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						

Csővonalas megvalósítás (MIC-3)

• 2. ciklus

- **swap1** második mikrolépése (ALU számol: $B-1$)
- **swap2** első mikrolépése (veremmutató értéke a B sínre)

	swap1	swap2	swap3	swap4	swap5	swap6
Ciklus	$MAR = SP - 1;$ rd	$MAR = SP$	$H = MDR;$ wr	$MDR = TOS$	$MAR = SP - 1;$ rd	$TOS = H;$ goto (MBR1)
1	$B = SP$					
2	$C = B - 1$	$B = SP$				
3						
4						
5						
6						
7						
8						
9						
10						
11						

Csővonalas megvalósítás (MIC-3)

• 3. ciklus

- **swap1** és **swap2** folytatása
- **swap3** nem indítható!
 - Meg kell várni **MDR**-t az első utasítás eredményeként! (RAW függőség)

Read After Write



	swap1	swap2	swap3	swap4	swap5	swap6
Ciklus	MAR = SP - 1; rd	MAR = SP	H = MDR; wr	MDR = TOS	MAR = SP - 1; rd	TOS = H; goto (MBR1)
1	B = SP					
2	C = B - 1	B = SP				
3	MAR = C; rd	C = B	Elakadás!			
4						
5						
6						
7						
8						
9						
10						
11						

Csővonalas megvalósítás (MIC-3)

- **4. ciklus**
 - **swap1** és **swap2** véget ér
 - **MDR** ezután válik majd használhatóvá

	swap1	swap2	swap3	swap4	swap5	swap6
Ciklus	MAR = SP - 1; rd	MAR = SP	H = MDR; wr	MDR = TOS	MAR = SP - 1; rd	TOS = H; goto (MBR1)
1	B = SP					
2	C = B - 1	B = SP				
3	MAR = C; rd	C = B	Elakadás!			
4	MDR = Mem	MAR = C	Elakadás!			
5						
6						
7						
8						
9						
10						
11						

Csővonalas megvalósítás (MIC-3)

- **5. ciklus**
 - Swap3 elindul

	swap1	swap2	swap3	swap4	swap5	swap6
Ciklus	MAR = SP - 1; rd	MAR = SP	H = MDR; wr	MDR = TOS	MAR = SP - 1; rd	TOS = H; goto (MBR1)
1	B = SP					
2	C = B - 1	B = SP				
3	MAR = C; rd	C = B				
4	MDR = Mem	MAR = C				
5			B = MDR			
6						
7						
8						
9						
10						
11						

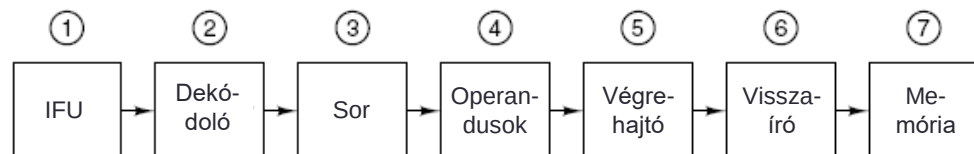
Csővonalas megvalósítás (MIC-3)

- **SWAP** utasítás mikrolépései MIC-3-on
 - Teljes kód

	swap1	swap2	swap3	swap4	swap5	swap6
Ciklus	MAR = SP - 1; rd	MAR = SP	H = MDR; wr	MDR = TOS	MAR = SP - 1; rd	TOS = H; goto (MBR1)
1	B = SP					
2	C = B - 1	B = SP				
3	MAR = C; rd	C = B				
4	MDR = Mem	MAR = C				
5			B = MDR			
6			C = B	B = TOS		
7			H = C; wr	C = B	B = SP	
8			Mem = MDR	MDR = C	C = B - 1	B = H
9					MAR = C; wr	C = B
10					Mem = MDR	TOS = C
11						goto (MBR1)

Hétszakaszú csővezeték (MIC-4)

- **Korábbi MIC architektúrák**
 - Nem értelmezik a bájtfolyamot
 - Nem tudják, hogy a következő bájtt utasítás vagy operandus
 - Nem ismerik az utasítás méretét
- **MIC-4 újdonságok**
 - Dekódoló egység
 - Bájtfolyam dekódolása utasításokként
 - Átalakítja mikroműveletek sorozatává egy ROM-táblázat alapján
 - Sor
 - Végrehajtáshoz sorba állítja a mikroműveleteket
 - **MIR1**, ..., **MIR4**
 - 4-7. szakaszok irányítására



Teljesítmény növelése

- **Megvalósítás javítása**

- PI. új CPU, memória beépítése az architektúra módosítása nélkül
 - Módosítás nélkül futnak a régi programok
- Nagyobb órajel
- CPU teljesítményének javítása
 - Gyorsítótár
 - Elágazásjövendölés
 - Sorrendtől eltérő végrehajtás regiszterátnevezéssel
 - Feltételezett végrehajtás

- **Architektúra tökéletesítése**

- Új utasítások, regiszterek
 - A régiek megtartásával a kompatibilitás megmarad
 - A szoftverek újrafordíthatók az újdonságok kihasználására

Gyorsítótár (Cache)

- **Probléma**

- A központi memória elérése lassú!
 - Pedig onnan töltődnek be az utasítások és az adatok!
- Gyors elérésű memória rendkívül drága!
 - És nem is érdemes a fizikai mérete miatt nagyot a CPU-ra integrálni
 - Kisebb méretű alkalmazása azért még kifizetődő
 - Jól kell tudni használni!

- **Lokalitási elv**

- *Térbeli lokalitás*
 - Az elkövetkezendő utasítások végrehajtásakor valószínűleg el kell érni a jelenlegi memóriacím után következő bájtokat
- *Időbeli lokalitás*
 - A nemrégiben elért memóriahelyeket esetleg újra el kell érni

- **Ötlet**

- Tároljuk ezeket a bájtokat gyors elérésű memóriában!
- Ha szükség van rájuk, onnan töltsük be, ne a központi memóriából

Gyorsítótár

- **Fogalmak**

- **Találati arány:** h

- Az összes hivatkozás mekkora hányada szolgálható ki a gyorsítótárból

- **Hiba arány:** $1-h$

- Ha a gyorsítótár elérési ideje: c , a memória elérési ideje: m , akkor
átlagos elérési idő = $c + (1-h) \cdot m$

- Szerencsétlen esetben lassabb lehet, mint a közvetlen memóriacímzés!

- **Gyorsítósor**

- Rögzített méretű blokkok a gyorsítótárban (pl. 4-64 bájt)
 - Teljes blokk mozgatásra kerül
 - Nagyobb méretű sor esetén
 - a hivatkozott cím nagyobb környezete lesz a gyorsítótárban, viszont
 - nagyobb a sor betöltési ideje is,
 - ugyanakkora tárból kevesebb gyorsító sor fér el

A gyorsítótár-elérés mindenképpen megtörténik + ha nem szolgálható ki onnan a kérés, akkor még a memória is dolgozik!

Gyorsítótár

- **Szétválasztott gyorsítótár**

- Külön utasításoknak és adatoknak
- Egyik végzi az utasítás (I), másik az operandus (D) előolvasást
- Az utasítás gyorsítótárat sohasem kell visszaírni
 - Az utasítások nem módosulnak

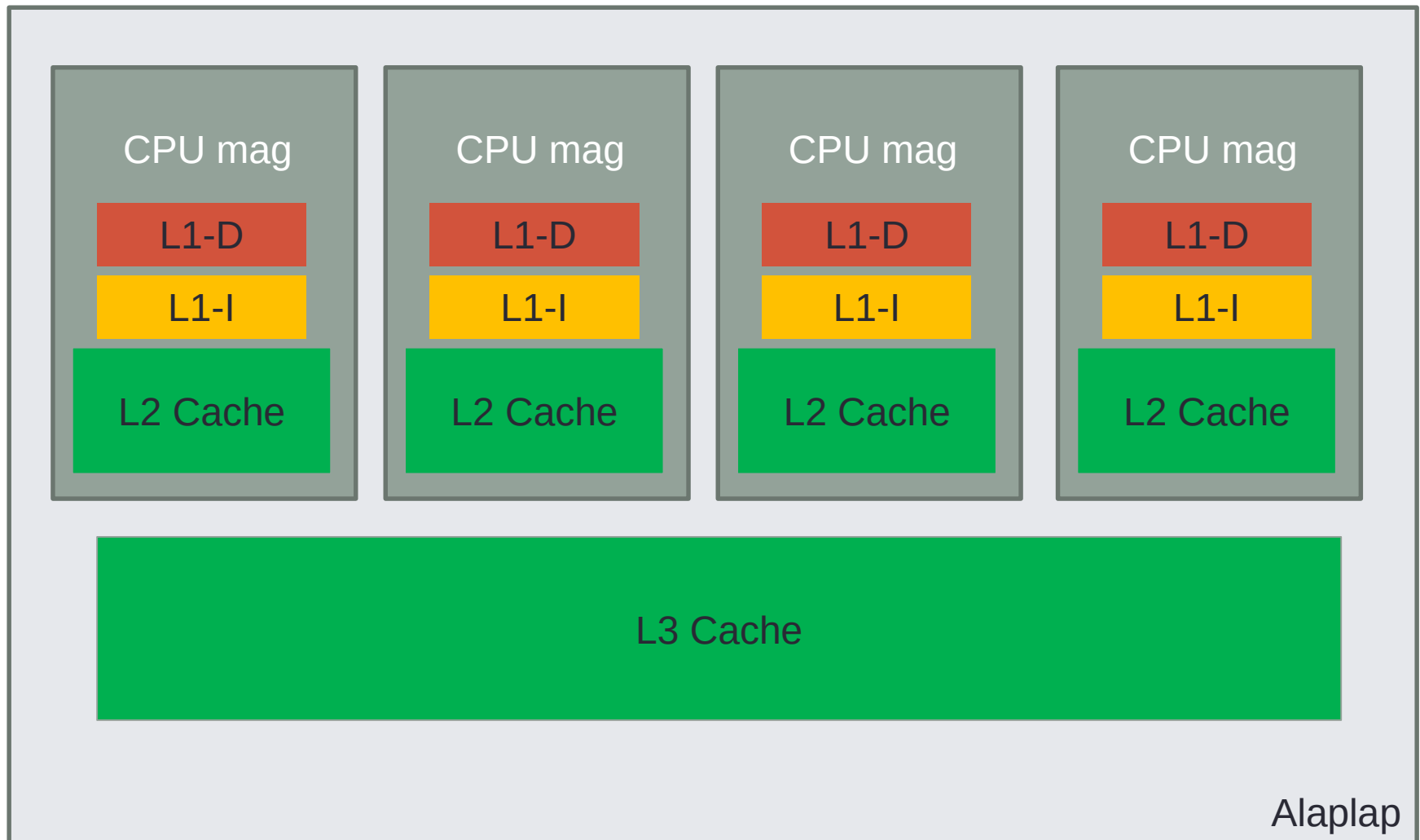
- **Egyesített gyorsítótár**

- Nem lehetséges párhuzamosítás

- **Hierarchia**

- Elsődleges gyorsítótár (L1), a CPU lapkán (KB nagyságrendű),
- Másodlagos gyorsítótár (L2), a CPU-val egy tokban (~MB),
- További szintek: külön tokban (néhány MB SRAM).

Gyorsítótár szintek

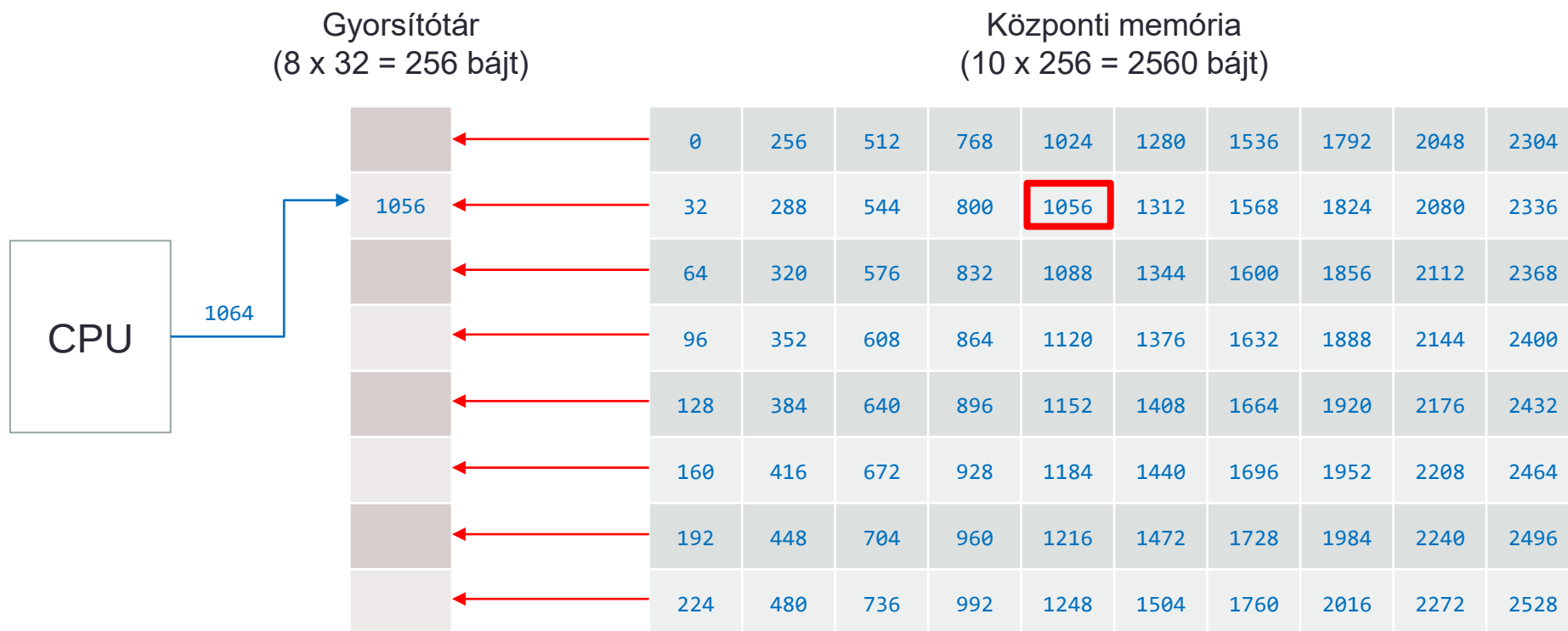


Gyorsítótár

- **Direkt leképezésű gyorsítótárak**

- A memóriacímből egyértelműen számítható, melyik bejegyzésnél kell keresni
 - Akár hardveresen huzalozható
 - Ha nincs ott, akkor nincs a gyorsítótárban
 - Jól meghatározható címtartományok ugyanarra a bejegyzésre kerülnek
 - Ütközés előfordulhat!
- Gyakran alkalmazott elv

Direkt leképezésű gyorsítótár (egyszerűsített példa)



- **Gyorsítótár:** kezdetben üres
- **CPU:** Szó beolvasása a központi memória 1064. cellájától kezdődően
- **Gyorsítótár:** 32 bájt olvasása az 1 indexű gyorsítósorba az 1056-1087 memóriatarományból
- **Gyorsítótár:** Szó olvasása a gyorsítósor 8 indexű cellájától kezdődően
- **CPU:** Szó olvasása a központi memória 1068. cellájától kezdődően
- **Gyorsítótár:** Szó olvasása az 1 indexű gyorsítósor 12 indexű cellájától kezdődően

- **CPU:** Szó beolvasása a központi memória 608. cellájától kezdődően
- **Gyorsítótár:** 32 bájt olvasása az 3 indexű gyorsítósorba a 608-639 memóriatarományból
- **Gyorsítótár:** Szó olvasása a gyorsítósor 0 indexű cellájától kezdődően
- **CPU:** szó olvasása a 824. cellától kezdődően
- **Gyorsítótár:** 1 indexű gyorsítósor eldobása, újraolvasás a 800-831 memóriatarományból
- **Gyorsítótár:** szó olvasása a gyorsítósor 24 indexű cellájától kezdődően

Direkt leképezésű gyorsítótár (összetettebb példa)

- **Bejegyzés részei** (2048 bejegyzéses, 32 bájtos gyorsítósor esetén)
 - *Valid bit*: érvényes-e az adat (kezdetben mindegyik hamis)
 - *TAG*: 16 bites egyedi érték. Adat származási helyét azonosítja.
 - *DATA*: 32 bájtot, a központi memóriában lévő adat másolata

Bejegyzés	Valid	TAG	DATA	Hozzá tartozó címtartomány (TAG alapján)
2047				
...	...	...	...	...
3				96-127, 65632-65663, 131168-131199, ...
2				64-95, 65600-65631, 131136-131167, ...
1				32-63, 65568-65599, 131104-131135, ...
0				0-31, 65536-65567, 131072-131103, ...

- 32 bites virtuális cím

Melyik szó
(4 bájtot) kell?

Szón belül melyik bájtot kell?

TAG (16 bit)	LINE (11 bit)	WORD (3 bit)	BYTE (2 bit)
--------------	---------------	--------------	--------------

Direkt leképezésű gyorsítótár (összetettebb példa)

- **Címzés menete**

- CPU beállítja a memóriacímet (32 bites virtuális cím)
- Hardver kiveszi a 11 **LINE** bitet, a táblázat indexelésére
 - $2^{11} = 2048$
- Táblázat bejegyzés **Valid** bitjének ellenőrzése
 - *Ha hamis*
 - Nincs a gyorsítótárban → központi memória címzés kell; gyorsítósor a gyorsítótárba (**DATA**), **TAG** és **Valid** kitöltése; adat szolgáltatása
 - *Ha igaz*
 - **TAG** mezők összehasonlítása
 - *Ha különböznek*
 - Más van a gyorsítótárban → központi memória címzés kell; ha változott az adattartalom, akkor visszaírás, majd betöltés, mint fent, a *hamis* ágon volt
 - *Ha egyeznek*
 - **WORD** és **BYTE** által megadott szó vagy bájt elérése

Példakód #1 a működésre

- **C példaprogram**

```
int array_A[16], array_B[16], array_C[16];  
...  
int idx;  
for(idx = 0, idx < 16; idx++)  
{  
    array_C[idx] = array_A[idx] + array_B[idx];  
}
```

- Feltesszük, hogy az `int` típus 32 bites egész értéket jelöl.
- Így a tömbök 64 bájt méretű memóriaterületet foglalnak.
- Vegyük észre, hogy ha a memóriában bájtfolytónosan helyezkednek el, akkor tömbönként 2-2 gyorsítósort foglalnak el, a gyorsítótárban egymás utáni sorokban.

Példakód #1 a működésre

• Lépések

- Feltesszük, hogy kezdetben üres a gyorsítótár.
- `idx = 0`
 - `array_A[0]` elérése
 - Mivel nincs a gyorsítótárban, ezért az első 32 bájtjának (8 tömb értékének) beolvasása a `0` indexű sorba.
 - `0` indexű szó `0.` bájtól beolvasásra
 - `array_B[0]` elérése
 - Mivel nincs a gyorsítótárban, ezért az első 32 bájtjának (8 tömb értékének) beolvasása a `2` indexű sorba.
 - `0` indexű szó `0.` bájtól beolvasásra
 - `array_C[0]` elérése
 - Mivel nincs a gyorsítótárban, ezért az első 32 bájtjának (8 tömb értékének) beolvasása a `4` indexű sorba.
 - Összeg írása a `0` indexű szó `0.` bájtjától

Bejegyzés	Valid	Tag	Data	Hozzá tartozó címtartomány (Tag alapján)
2047				
...	...	...	...	...
3				96-127,
2				64-95, <code>array_B[0-7]</code>
1				32-63,
0				0-31, <code>array_A[0-7]</code>

Példakód #1 a működésre

• Lépések

• `idx = 1`

• `array_A[1]` elérése

- Elérhető a gyorsítótárban: `1` indexű szó `0.` bájtól beolvasásra

• `array_B[1]` elérése

- Elérhető a gyorsítótárban: `1` indexű szó `0.` bájtól beolvasásra

• `array_C[1]` elérése

- Elérhető a gyorsítótárban: Összeg írása az `1` indexű szó `0.` bájtjától

• Nagyobb `idx` értékek esetén

- `idx = 0, ..., 7` kiszolgálható a `0, 2` és `4` sorokból

- `idx = 8` esetén a következő 32 bájtok beolvasódnak az `1, 3` és `5` gyorsítósorokba

- `idx = 9, ..., 15` esetén már mindhárom tömb teljes egészében a gyorsítótárban van, onnan kiszolgálható

Bejegyzés	Valid	Tag	Data	Hozzá tartozó címtartomány (Tag alapján)
2047				
...	...	...	...	...
3				96-127, <code>array_B[8-15]</code>
2				64-95, <code>array_B[0-7]</code>
1				32-63, <code>array_A[8-15]</code>
0				0-31, <code>array_A[0-7]</code>

Példakód #2 a működésre

- **C példaprogram**

```
int array_A[16384], array_B[16384], array_C[16384];  
...  
int idx;  
for(idx = 0, idx < 16384; idx++)  
{  
    array_C[idx] = array_A[idx] + array_B[idx];  
}
```

- Feltesszük, hogy az `int` típus 32 bites egész értéket jelöl.
- Így a tömbök 64 kB méretű memóriaterületet foglalnak.
- Vegyük észre, hogy ha a memóriában bájt folytonosan helyezkednek el, akkor ugyanarra a gyorsítósorra kerülnek az azonos tömbindexű elemek, ami folyamatos ütközést fog okozni!

Példakód #2 a működésre

Bejegyzés	Valid	Tag	Data	Hozzá tartozó címtartomány (Tag alapján)
2047				
...	...	...	...	...
3				96-127, 65632-65663, 131168-131199, ...
2				64-95, 65600-65631, 131136-131167, ...
1				32-63, 65568-65599, 131103-131135, ...
0				0-31, 65536-65567, 131072-131103, ...

• Lépések

- Feltesszük, hogy kezdetben üres a gyorsítótár.
- `idx = 0`
 - `array_A[0]` elérése (0 memóriacím index)
 - Mivel nincs a gyorsítótárban, ezért az első 32 bájtnak (8 tömb értékének) beolvasása a 0 indexű sorba.
 - 0 indexű szó 0. bájtól beolvasásra
 - `array_B[0]` elérése (65536 memóriacím index)
 - Van adat a gyorsítótárban, de az az `array_A` első 32 bájtja (TAG alapján ütközés van) ezért az első 32 bájtnak (8 tömb értékének) beolvasása a 0 indexű sorba, felülírva `array_A` értékeit!
 - 0 indexű szó 0. bájtól beolvasásra
 - `array_C[0]` elérése (131072 memóriacím index)
 - Van adat a gyorsítótárban, de az az `array_B` első 32 bájtja (TAG alapján ütközés van) ezért az első 32 bájtnak (8 tömb értékének) beolvasása a 0 indexű sorba, felülírva `array_B` értékeit!
 - Összeg írása a 0 indexű szó 0. bájtyától
- Nagyobb `idx` értékek
 - **Mindig ütközés van, folyamatos gyorsítósor olvasás kell!**
 - `array_A` olvasáskor ráadásul ki kell írni a memóriába a korábbi `array_C` értékeket!

Gyorsítótár példák következményei

- **A gyakorlatban nagyon hatékonyan működik**
 - Általános célú gépen (PC, okostelefon), általános célú feladatokra
 - De pl. mikrovezérlőkben rendszerint nincs szükség ilyenre
- **A példaprogramok kapcsán**
 - Magasszintű nyelveken programozva nem szükséges a gyorsítótár tényleges működésének ismerete
 - De segíthet megérteni a futásidőbeli különbségeket
 - Nagy méretű memóriablokkok használatakor mi történik?
 - A ciklusok magjának kódja beleférjen a gyorsítótárba
 - Profilozást használhatunk a kiderítésére
 - Program hatékonyság időbeli vizsgálata

Gyorsítótár típusok

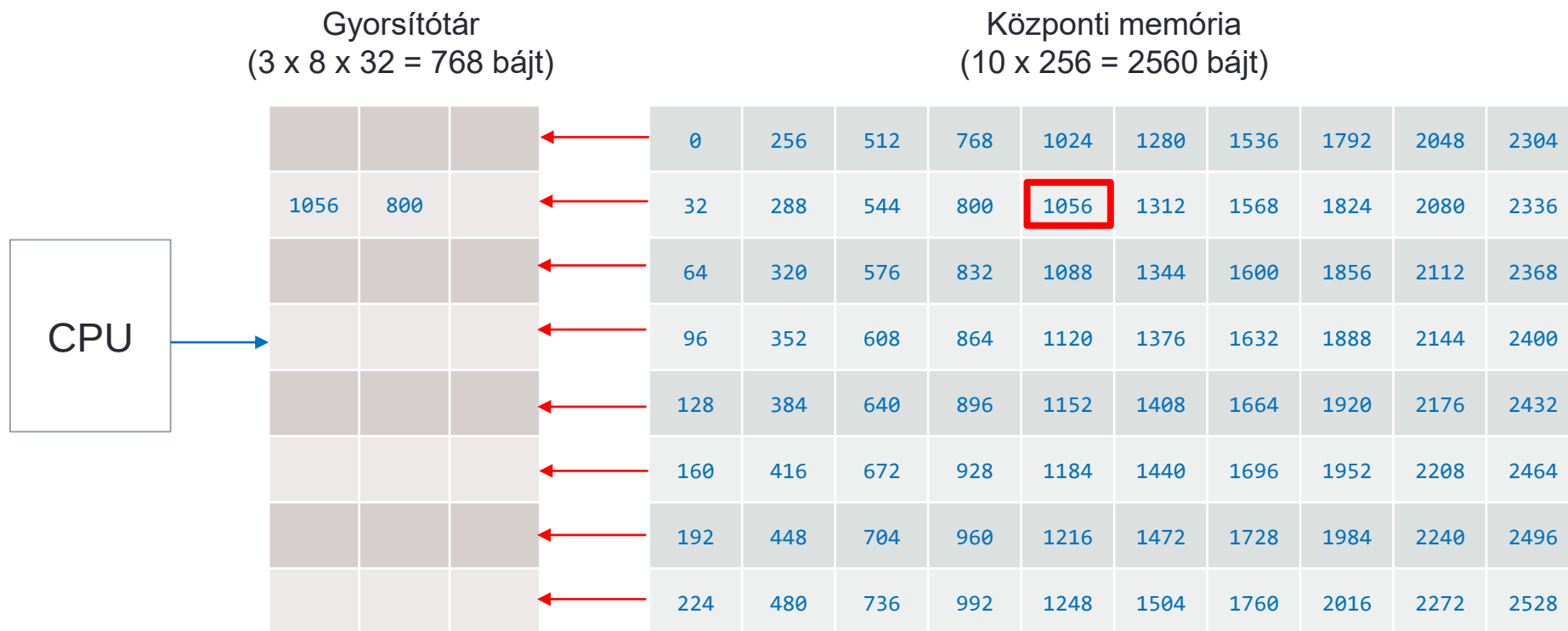
- **Direkt leképezésű problémái**

- 1 bejegyzéshez a címtartomány több része tartozik
- Ezek közül csak 1 lehet a gyorsítótárban
- Ha másikra történik hivatkozás, cserélni kell
 - Gyakori csere gyenge teljesítményhez vezet!
 - Pl. Fordítóprogramok optimalizálhatják az adatok memóriacímét
 - Szétválasztott gyorsítótár esetén kisebb probléma

- **Halmazkezelésű (csoportasszociatív) gyorsítótár**

- Két vagy több gyorsítósor is tartozhat 1 táblázat bejegyzéshez
 - Új feladat a bejegyzéshalmaz nyilvántartása
- n utas halmazkezelésű gyorsítótár: n darab lehetséges bejegyzés
- Bonyolultabb áramkört igényel, de megéri
 - Lehet pl. 2 vagy 4 utas
 - Akár 2048 utas is (nem feltétlenül célszerű)

3-utas halmazkezelésű gyorsítótár példa



- Egy gyorsítósor bejegyzés a hozzá tartozó címtartományból 3 darab 32 bájtos területet tud tartalmazni
- Korábbi egyszerű példa esete
 - Elfér az **1056-1087** és a **800-831** tartomány is

Halmazkezelésű gyorsítótár

- **Új probléma**

- Ha egy bejegyzéshez betelt minden gyorsítósor, melyiket kell eldobni?

- **Megoldási lehetőség**

- Legrégebben használt eldobása (LRU – *Least Recently Used*)
 - Rendezett lista nyilvántartása szükséges minden bejegyzéshalmazhoz

Teljes leképezésű gyorsítótár

- **Bármelyik cím bármelyik bejegyzéshez tartozhat**
 - Kereséskor minden bejegyzés **TAG** mezőjével össze kell hasonlítani
 - Nagyon költségesen valósítható meg, a hardver erős melegedésre hajlamos a folyamatos kihasználtságtól
 - Gyakorlatban nem (igazán) alkalmazzák

Gyorsítótárak írási problémája

- **Mi történjen, ha memóriába íráskor az adat a *gyorsítótárban van?***
 - Eldobhatjuk a bejegyzést és az adatot a főmemóriába írhatjuk, vagy
 - Frissíthetjük a gyorsítótárban (ez a jellemzőbb)
 - **Ha a gyorsítótárban frissítünk, mikor kerüljön ki a főmemóriába?**
 - **Írásáteresztés:** az adat azonnal kiírásra kerül a főmemóriába is
 - Egyszerűbb, megbízhatóbb, de nagyobb írásforgalom
 - **Késleltetett írás:** akkor kerül a főmemóriába, ha a gyorsítósor eldobásra kerül
- **Mi történjen, ha memóriába íráskor az adat *nincs a gyorsítótárban?***
 - **Írjunk csak a főmemóriába**
 - Írásáteresztő stratégiakor jellemző
 - **Írásallokálás:** töltsük be az adatot a gyorsítótárba
 - Késleltetett írás stratégia esetén jellemző
 - Csak akkor előnyös, ha ismételt írások történnek egy soron belül

Gyorsítótár példák

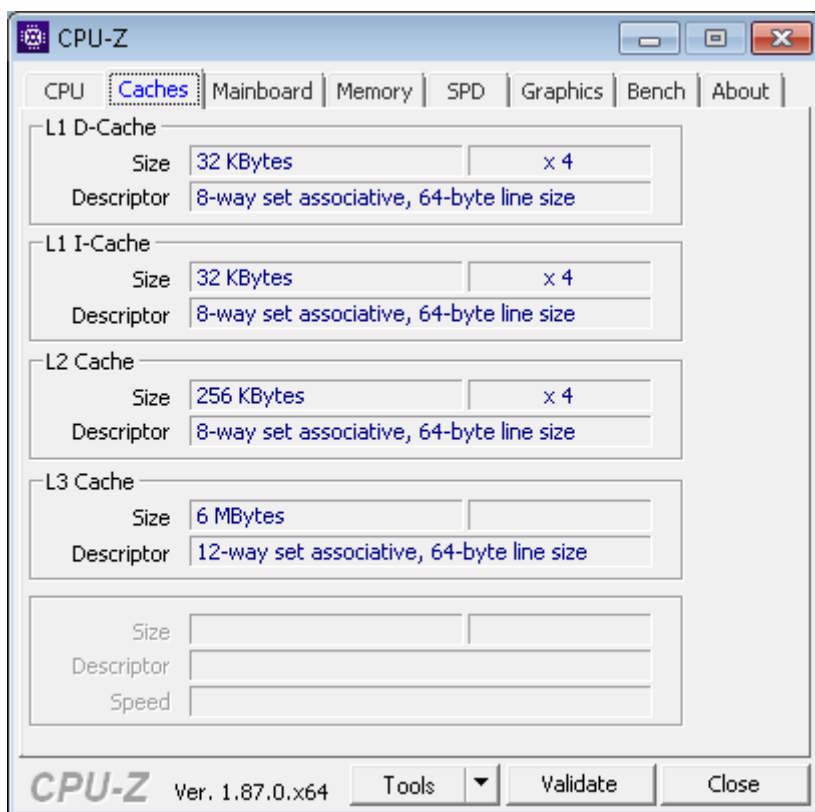
- **CPU-Z**

- Windows rendszeren használható diagnosztikai program
- Leírást ad a CPU cache adatokról is

- **Lapkagyártók leírásai**

- <https://www.intel.com/content/www/us/en/products/processors.html>
- <https://www.intel.com/content/www/us/en/products/processors/core.html>
- <https://www.amd.com/en/products/processors-desktop>
- <https://www.amd.com/en/products/processors-laptop>
- <https://www.amd.com/en/processors/workstation>
- <https://www.amd.com/en/products/embedded>
- <https://en.wikichip.org/>

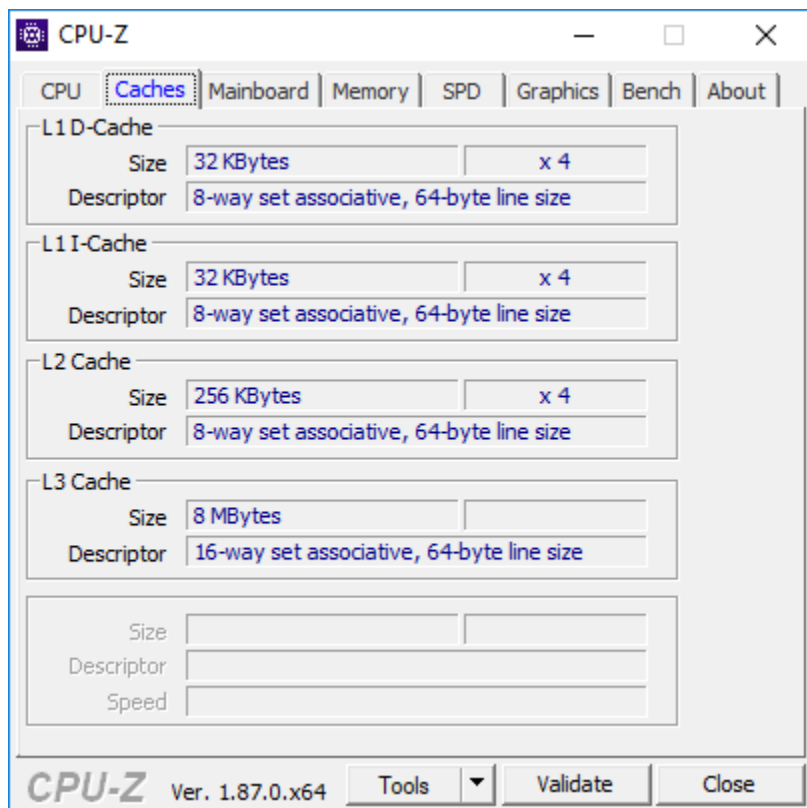
Intel Core i5-2500 (2011 Q1)



- **L1-adat**
 - 4x32 KB, 8-utas
- **L1-utasítás**
 - 4x32 KB, 8-utas
- **L2**
 - 4x256 KB, 8-utas
- **L3**
 - 6 MB, 12-utas

Mindegyik halmazkezelésű, 64 bájtos gyorsítósorral.

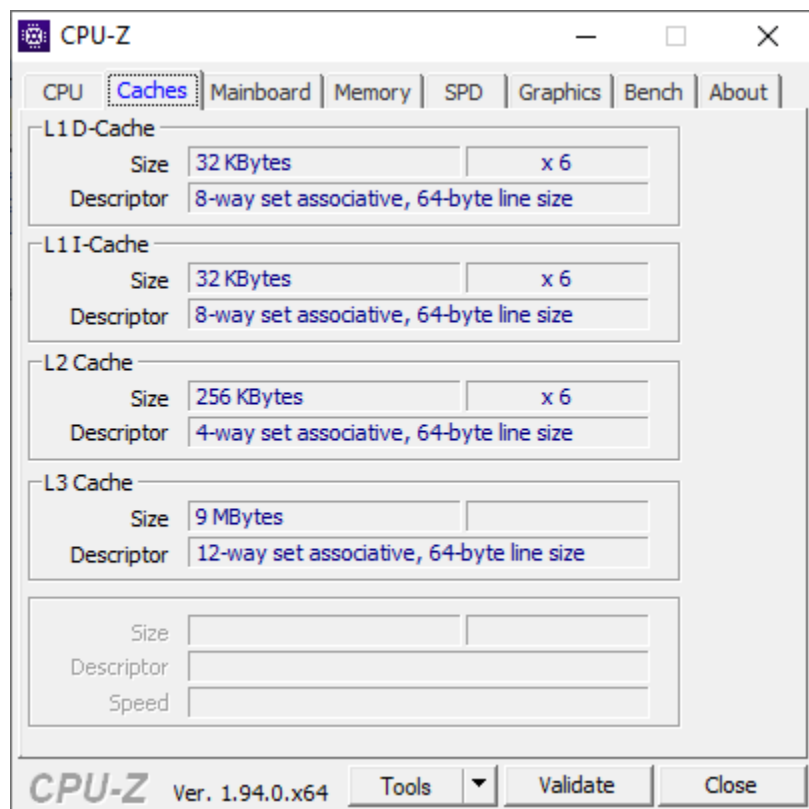
Intel Core i7-4770 (2013 Q2)



- **L1-adat**
 - 4x32 KB, 8-utas
- **L1-utasítás**
 - 4x32 KB, 8-utas
- **L2**
 - 4x256 KB, 8-utas
- **L3**
 - 8 MB, 16-utas

Mindegyik halmazkezelésű, 64 bájtos gyorsítósorral.

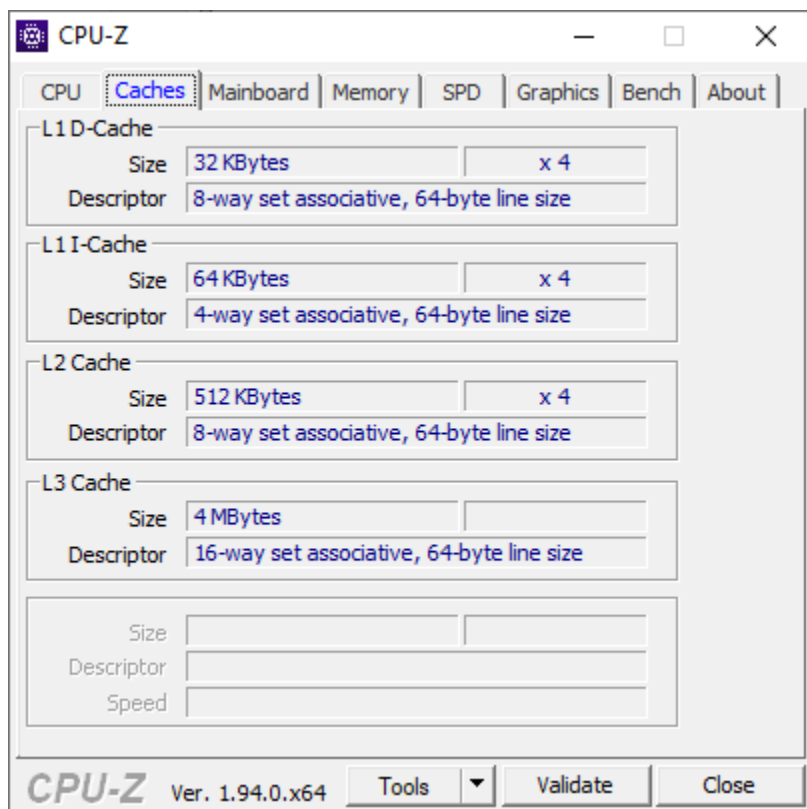
Intel Core i5-9400 (2019 Q1)



- **L1-adat**
 - 6x32 KB, 8-utas
- **L1-utasítás**
 - 6x32 KB, 8-utas
- **L2**
 - 6x256 KB, 4-utas
- **L3**
 - 9 MB, 12-utas

Mindegyik halmazkezelésű, 64 bájtos gyorsítósorral.

AMD Ryzen 7 Mobile 3700U (2019 Q1)



- **L1-adat**
 - 4x32 KB, 8-utas
- **L1-utasítás**
 - 4x64 KB, 4-utas
- **L2**
 - 4x512 KB, 8-utas
- **L3**
 - 4 MB, 16-utas

Mindegyik halmazkezelésű, 64 bájtos gyorsítósorral.

Qualcomm Snapdragon 865 (2019 Q4)

- **Elsődleges Cortex A77**

- L1-adat: 1x64 KB, 4-utas
- L1-utasítás: 1x64 KB, 4-utas
- L2: 1x512 KB 8-utas

- **3 magos Cortex A77 csoport**

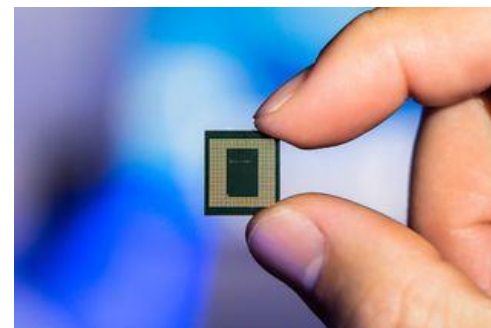
- L1-adat: 3x64 KB 4-utas
- L1-utasítás: 3x64 KB, 4-utas
- L2: 3x256 KB, 8-utas

- **4 magos Cortex A55 csoport**

- L1-adat: 4x64 KB, 4-utas
- L1-utasítás: 4x6 KB, 2-utas
- L2: 4x128 KB, 8-utas

- **Globális**

- L3: 4 MB



Elágazásjövendölés

- **Csővezeték architektúra problémái**

- Utasítás végrehajtás részekre bontva, egyszerre több is végrehajtás alatt áll párhuzamosan
- Lineáris kóddal dolgoznak a legjobban
- A gyakorlatban sok *feltételes* vagy *feltétel nélküli* elágazás van!

if(i == 0)	CMP i,0
k = 1;	BNE Else
else	Then: MOV k,1
k = 2;	BR Next
	Else: MOV k,2
	Next: ...

C nyelvű programrészlet

Assembly kód

- **Mire kiderül, hogy hova kell ugrani, a rákövetkező utasítás végrehajtása már megkezdődött a csővezetéken!**
 - Ezt hogyan tudjuk „visszavonni” vagy „hatástalanítani”?
 - Meg tudjuk-e tippelni egy feltételes elágazásnál, hogy melyik ág teljesül majd?

Elágazásjövendölés

- **Feltétel nélküli ugrás problémája**
 - Tudjuk hova kell ugrani, de a cím csak a csővezeték második szakaszában derül ki!
 - Dekódoláskor
 - A rákövetkező utasítás végrehajtása ekkor már elkezdődött!
 - UltraSPARC III
 - **Végrehajtódik a rákövetkező utasítás!!!**
 - **Eltolási rés:** elágazás utáni tartomány, amely végrehajtódik (leggyakrabban **NOP** (hatás nélküli utasítás) kerül ide)
 - Pentium 4
 - Bonyolult módon, de kikerüli a végrehajtást

Elágazásjövendölés

- **Feltételes ugrás problémája**

- Itt is létrejönnek eltolási rések, és ráadásul
- a feltétel kiértékelése a csővezeték egy még későbbi szakaszán történik meg. Ha ezt kivárjuk, **bedugulás** következik be.
- **Jövendölés**: próbáljuk kitalálni, melyik ág következik be, és azt kezdjük el végrehajtani

- **Mi történjen jó jövendölés esetén?**

- Szuper! Nincs tennivaló!

- **Mi történjen rossz jövendölés esetén?**

- Engedjük az utasítások végrehajtását, amíg nem változtatják meg a gép állapotát
 - Pl. amíg nem ír regisztert/memóriát
 - Vagy használjunk **rejtett firkáló** regisztert
 - Csak akkor kerül az érték az „igaziba”, ha ez a jó ág
- Jegyezzük fel a módosuló regiszterek értékét, hogy visszaállítható legyen, ha rossz a jövendölés

Jövendölési stratégiák

- **Egyszerű stratégia**

- Minden *visszafelé történő* feltételes elágazás bekövetkezik
 - Pl. ciklus végi visszaugrások
- Egyetlen *előreirányuló feltételes* elágazás sem következik be
 - Jó a tipp pl. hibakezelő ágak esetén (ha hiba van, átugrunk egy kódrészt)

- **Előnye**

- Egyszerűen alkalmazható

- **Hátránya**

- Nem túl jó, de jobb mint semmi

Jövendölési stratégiák

- **Dinamikus elágazásjövendölés**

- Feljegyezzük, hogy adott memóriacímen milyen irányba történt elágazás legutóbb, és ezt vesszük
- Előzménytábla: gyorsítótárhoz hasonló felépítés
 - Valid bit, elágazási cím, igaz/hamis legutóbb
 - n -utas megvalósítás is megoldható
- Kiegészíthető úgy, hogy csak a második más irányú ág után változzon
 - Ciklusok esetén kilépés után és újbóli belépéskor hasznos

- **Előnye**

- Futási időben működik
- Alkalmazkodik a program viselkedéséhez

- **Hátránya**

- Speciális és drága hardver szükséges

Jövendölési stratégiák

- **Statikus elágazásjövendölés**

- Fordítóprogram ad segítséget
 - Pl.: `for( i = 0; i < 1000000; i++ ) { ... }` esetén a ciklus végi ugrást szinte mindig végre kell hajtani
 - Megfelelő utasításrendszer (ISA szint) szükséges
 - UltraSPARC III: **jövendölő elágazó utasítások**
 - Az utasítás kódja tartalmazza a jövendölést, amit a fordítóprogram tölt ki
 - „ugrás, ha a feltétel teljesül – és várhatóan teljesülni fog”
 - „ugrás, ha a feltétel teljesül – de várhatóan nem fog teljesülni”
- Megfigyelésen alapuló stratégia (profilozás)
 - A programot futtatjuk
 - Közben figyeljük az elágazások viselkedését
 - Ezt betápláljuk a fordítóprogramba, ami előállítja a jövendölő elágazó utasításokat

Sorrendtől eltérő végrehajtás

- **Modern CPU-k**

- Csővezetékesek és szuperskalárisak
- Egy ciklusban akár több utasítás végrehajtását is elindíthatják
 - Akár 4-6 utasítás is indulhat!
- Elakadás következik be, ha egy utasítás egy megelőző eredményére vár (függő utasítás; lásd a korábbi dia példáját!)
 - **RAW** (Read After Write): olyan adatot olvasnánk, amit egy előző még ír
 - **WAR** (Write After Read): olyan helyre írnánk, amelyről egy előző még olvas
 - **WAW** (Write After Write): olyan helyre írnánk, amelyet egy előző még ír

Regiszterátnevezés

- **Bizonyos WAR és WAW függőségek kiküszöbölése**
 - Függőség esetén az eredetileg kért regiszter helyett egy titkos regiszter kerül felhasználásra
 - Akár több tucat titkos regisztere is lehet a CPU-nak!
 - Táblázatot tart nyilván a leképezések nyomon követésére
 - A függőség megszűnése után
 - az eredmény bekerülhet az eredetileg megnevezett regiszterbe, vagy
 - a további utasítások erre a titkos regiszterre vonatkozhatnak
- **RAW függőség így nem kiküszöbölhető**
 - Mindenképpen meg kell várni az előző író utasítás befejeződését, mert az szolgáltatja az olvasandó adatot

Sorrendtől eltérő végrehajtás

- Ha alkalmasak egy ciklusban több utasítás indítására, akkor **célszerű átugrani a függő utasításokat** (rákövetkezőkkel folytatni)
 - Lineáris (elágazás nélküli) kódblokkon belül működik
 - **Vigyázni kell!**
 - Ugyanazt az eredményt kell szolgáltatni, mint a sorrendben történő végrehajtásnak!
 - Függőségeket kezelni!
- **Sorrend szerint befejezés megkövetelése**
 - Fontos lehet például a megszakítások kezelése miatt
 - Nehéz lenne visszaállítani a megszakítás bekövetkezte előtti állapotot
 - Ez a „**pontos megszakítás**” tulajdonság

Feltételezett végrehajtás

• Programok

- Alapblokkok: lineáris kódsorozat, vezérlési szerkezet nélkül
- Ezeket összekötő vezérlési szerkezetek

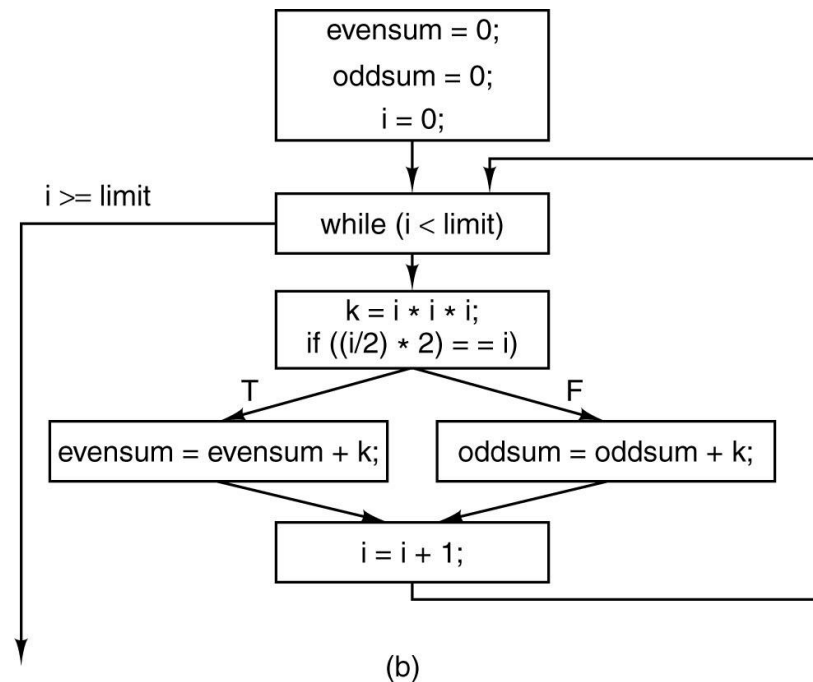
• Végrehajtunk olyan kódot, amiről még nem tudjuk, hogy szükség lesz-e rá

```

evensum = 0;
oddsum = 0;
i = 0;
while (i < limit) {
    k = i * i * i;
    if (((i/2) * 2) == i)
        evensum = evensum + k;
    else
        oddsum = oddsum + k;
    i = i + 1;
}

```

(a)



(b)

Feltételezett végrehajtás

- **Megengedjük az alablokkok határain átnyúló végrehajtás-sorrendi átrendezéseket**
 - *Emelésnek* nevezzük
 - Fordítóprogram közreműködése is kell
- **Felmerülő kérdések**
 - Nem lehetnek visszavonhatatlan eredmények
 - Pl. regiszter átnevezések használatával
 - Mi történik, ha a feltételesen végrehajtott utasítás kivételt okoz?
 - Mi történik, ha gyorsítótárhiányt okoz?
 - Spekulatív betöltő utasítás: ha elérhető a gyorsítótárban, akkor betölt, egyébként feladja
 - Mi történik, ha csapdát vált ki?
 - `if(x > 0) z = y / x; // nem lehet(ne) 0-val való
// osztás`
 - Mérgezés-bit: csak akkor okoz csapdát, ha erre az ágra kerül a vezérlés

CPU sérülékenységek

- **Meltdown, Spectre**

- 2018. január: A modern processzorok mikroarchitektúra tervezési hibái (elsősorban Intel, de részben ARM is): egyik processzus hozzáférhet egy másik adataihoz
- Javítása a kapcsolódó funkciók (például HyperThreading) kikapcsolásával → teljesítménycsökkenés! (Nem szoftveres probléma, nem frissíthető!)
 - Vagy cserélni kell újabbra a CPU-t, ahol ezt már javították...
- [https://en.wikipedia.org/wiki/Meltdown_\(security_vulnerability\)](https://en.wikipedia.org/wiki/Meltdown_(security_vulnerability))
- https://hvg.hu/tudomany/20180108_meltdown_spectre_hiba_mit_jelent_szoftverfrisites_javitas_telepitese_virusirto_antivirus

- **MDS (Microarchitectural Data Sampling)**

- https://hvg.hu/tudomany/20190516_intel_processzor_kiberbiztonsag_hacker_micro_architectural_data_sampling_mds
- <https://www.youtube.com/watch?v=0eb-04yKK2c>

- **L1DES**

- https://hvg.hu/tudomany/20200129_intel_processzor_sebezhetoseg

- **Mikro-op gyorsítótár sérülékenység**

- https://hvg.hu/tudomany/20210503_processzor_amd_intel_spectre_kiberbiztonsag_sebezhetoseg

- **És még hol a vége...**

Mikroarchitektúra példák

- Csak érdekesség, nem kell megtanulni!

	Pentium 4	UltraSPARC III cu	8051
Típus	Asztali PC	Asztali PC	Beágyazott rendszerek
Utasításrendszer	CISC, kevés regiszter, változó hosszúságú utasítások, de belső csővezetékes RISC mag	RISC, a gépi utasítások mikroműveletek, általában 4 bájtos utasítások	Utasítások általában 1 órajelciklus alatt végrehajtnak, speciális célú 8 és 16 bites regiszterek
Gyorsítótár	L2: belső, 1 MB, 8 utas, halmazkezelésű, visszaíró, 128 bájtos gyorsítósor L1 adat: 4 utas, halmazkezelésű, 64 bájtos gyorsítósor, írásáteresztő L2-be L1 kód: 12K mikroművelet	L2: külső, 1, 2, 8 MB L1 adat: 4 utas halmazkezelésű, 32 bájtos gyorsítósor L1 kód: 4 utas halmazkezelésű, 2-2 KB előre betöltő és tároló gyorsítótárak	Nincs
Utasításbetöltő	L2 gyorsítótárból dekódol CISC-ből RISC mikroműveletekre	Akár 4 utasítás előkészítése ciklusonként	
Végrehajtás	Sorrend szerinti kiosztás, sorrendtől eltérő végrehajtás, sorrend szerinti befejezés, regiszterátnevezés	Sorrend szerinti kiosztás, sorrendtől eltérő végrehajtás, sorrend szerinti befejezés; regiszterátnevezés	Sorrend szerinti kiosztás, sorrend szerinti végrehajtás, sorrend szerinti befejezés
Elágazásjövendölés	L1 gyorsítótárban 4096 utolsó elágazás tárolása, Ha nincs benne, egyszerű jövendölést használ	Van. A feltételes utasítás utáni utasítást mindenképpen végrehajtja!	Nincs
Egyéb	MIC-4-hez hasonló		Nincs csővezeték MIC-1-szerű

Mikroarchitektúra példák

- Csak érdekesség, nem kell megtanulni!

	Intel Core i7	TI OMAP4430	ATmega168
Típus	Asztali PC	Mobil eszközök, táblagépek	Beágyazott rendszerek
Utasításrendszer	CISC, kevés regiszter, változó hosszúságú utasítások, de belső csővezetékes RISC mag	RISC, a gépi utasítások mikroműveletek, általában 4 bájtosak, új grafikus és multimédiás utasítások	Utasítások általában 1 órajelciklus alatt végrehajtnak, speciális célú 8 bites regiszterek
Gyorsítótár	L3: közös 1-20 MB, 12 utas, halmazkezelésű, 64 bájtos sor, L2: magonkénti belső, 256 KB, 8 utas, halmazkezelésű, visszaíró, 64 bájtos gyorsítósor L1: 32 KB, 8 utas, halmazkezelésű, 64 bájtos gyorsítósor, írásáteresztő L2-be	L2: 1 MB L1 adat: 32 KB, 4 utas halmazkezelésű, 32 bájtos sor L1 kód: 32 KB, 4 utas halmazkezelésű, 32 bájtos sor kis méretű ciklusoknak külön gyorsítótár	Nincs
Utasításbetöltő	Mikroutasítások gyorsítótárazása a dekódolás után (L0 tár)	Akár 4 utasítás előkészítése ciklusonként	Csővezetékes betöltés (2 lépéses)
Végrehajtás	Sorrend szerinti kiosztás, sorrendtől eltérhető végrehajtás, sorrend szerinti befejezés, regiszterátnevezés	11 elemű csővezeték; sorrendtől eltérhető végrehajtás, sorrend szerinti befejezés, regiszterátnevezés	Sorrend szerinti kiosztás, sorrend szerinti végrehajtás, sorrend szerinti befejezés
Elágazásjövendölés	Nagyon összetett, de algoritmus nem publikus	4K méretű elágazásjövendölő tábla, 1K méretű címtábla	Nincs
Egyéb	MIC-4-hez hasonló		MIC-1-szerű

Hibalista

- Tanenbaum könyv (2. magyar nyelvű kiadás)
 - 4.26 ábra (Háromsínes kód ILOAD végrehajtására)
 - iload1 lépésben még nem áll rendelkezésre MBRU!